Punto 1.

```python
import numpy as np
from scipy.optimize import linprog

# Queremos maximizar 180x1 + 220x2.
# Como linprog minimiza, hay qie usar valores negativos.
c = [-180, -220]

# Coeficientes de las Restricciones (Lado izquierdo: Matriz A)
# Las filas son Compuesto A, B y C respectivamente.
A = [
    [3, 4], # 3x1 + 4x2 (Compuesto A)
    [2, 1], # 2x1 + 1x2 (Compuesto B)
    [5, 3]  # 5x1 + 3x2 (Compuesto C)
]

# Disponibilidad (Lado derecho: Vector b)
b = [120, 80, 150]

# Límites de las variables (No negatividad: x >= 0)
x_bounds = (0, None)

# Resolver el problema
# Usamos el método 'highs'
res = linprog(c, A_ub=A, b_ub=b, bounds=[x_bounds, x_bounds], method='highs')

# Imprimir los resultados
print("-" * 30)
print("Punto 1. Mezcla de fertilizantes.")
print("-" * 30)

if res.success:
    print(f"Cantidad de F1 (x1) a producir: {res.x[0]:.2f} toneladas")
    print(f"Cantidad de F2 (x2) a producir: {res.x[1]:.2f} toneladas")
    print("-" * 30)
    print(f"GANANCIA TOTAL MÁXIMA: ${-res.fun:.2f}")
else:
    print("No se pudo encontrar una solución óptima.")
```

```
------------------------------
Punto 1. Mezcla de fertilizantes.
------------------------------
Cantidad de F1 (x1) a producir: 21.82 toneladas
Cantidad de F2 (x2) a producir: 13.64 toneladas
------------------------------
GANANCIA TOTAL MÁXIMA: $6927.27
```

Punto 2.

```python
import numpy as np

print("-" * 50)
print("Punto 2. Producción semanal de una fabrica.")
print("-" * 50)

# Cálculo de semanas 1 y 2
s = [200]
m = [80]

for i in range(2):
    s_next = 0.6 * s[i] + 0.2 * m[i] + 40
    m_next = 0.1 * s[i] + 0.5 * m[i] + 20
    s.append(s_next)
    m.append(m_next)
    print(f"Semana {i+1}: Sillas = {s_next}, Mesas = {m_next}")

# Cálculo del equilibrio
# 0.4S - 0.2M = 40
# -0.1S + 0.5M = 20
A = np.array([[0.4, -0.2], [-0.1, 0.5]])
B = np.array([40, 20])

equilibrio = np.linalg.solve(A, B)
print(f"\nValores de equilibrio:")
print(f"Sillas (S): {equilibrio[0]:.2f}")
print(f"Mesas (M): {equilibrio[1]:.2f}")
```

```
--------------------------------------------------
Punto 2. Producción semanal de una fabrica.
--------------------------------------------------
Semana 1: Sillas = 176.0, Mesas = 80.0
Semana 2: Sillas = 161.6, Mesas = 77.6

Valores de equilibrio:
Sillas (S): 133.33
Mesas (M): 66.67
```

Punto 3.

```python
import numpy as np
from scipy.integrate import solve_ivp

print("-" * 30)
print("Punto 3. Epidemia en una ciudad.")
print("-" * 30)

# Parámetros
beta = 0.00003
gamma = 0.1
P = 10000

# Modelo (EDO)
def modelo_sis(t, I):
    dIdt = beta * (P - I) * I - gamma * I
    return dIdt

# Condición inicial
I0 = [50]
t_i = 0
t_f = 150
t_eval = np.linspace(t_i, t_f, 100)

# Resolver
res = solve_ivp(modelo_sis, [t_i, t_f], I0, t_eval=t_eval)

# Extraer los datos de 'res'
t = res.t
I = res.y[0] # Es la primera fila de resultados

# Imprimir valores
print(f"Infectados iniciales:   {I0[0]}")
print(f"Valor máximo alcanzado: {max(I):.1f} personas (día {t[np.argmax(I)]:.1f})")
print(f"Valor al final del periodo: {I[-1]:.1f} personas")
print(f"Equilibrio teórico I*:  {P - gamma/beta:.1f} personas")
```

```
------------------------------
Punto 3. Epidemia en una ciudad.
------------------------------
Infectados iniciales:   50
Valor máximo alcanzado: 6671.5 personas (día 97.0)
Valor al final del periodo: 6666.4 personas
Equilibrio teórico I*:  6666.7 personas
```

Punto 4.

```python
import numpy as np
from scipy.optimize import fsolve

print("-" * 30)
print("Punto 4. Modelo logistico de pesca.")
print("-" * 30)

# Parámetros
r = 0.4
K = 1000
H = 80
P0 = 400

def modelo_pesca(Pn):
    # Ecuación: Pn + r * Pn * (1 - Pn/K) - H
    return Pn + r * Pn * (1 - Pn/K) - H

P1 = modelo_pesca(P0)
P2 = modelo_pesca(P1)

print("-" * 40)
print(f"Población al final del Año 1: {P1:.2f} peces")
print(f"Población al final del Año 2: {P2:.2f} peces")
print("-" * 40)

# Puntos de Equilibrio
# r * P * (1 - P/K) - H = 0
def ecuacion_equilibrio(P):
    return r * P * (1 - P/K) - H

# fsolve de scipy para encontrar las raíces de la ecuación
# dos "estimaciones" iniciales porque es una ecuación cuadrática
equilibrio_alto = fsolve(ecuacion_equilibrio, 800)[0]
equilibrio_bajo = fsolve(ecuacion_equilibrio, 200)[0]

print(f"Punto de equilibrio (Estable)):   {equilibrio_alto:.2f} peces")
print(f"Punto de equilibrio (Inestable): {equilibrio_bajo:.2f} peces")
print("-" * 40)

if P0 > equilibrio_bajo:
    print("Interpretación: La población sobrevivirá y tenderá al equilibrio estable.")
else:
    print("Interpretación: La población está por debajo del umbral y se extinguirá.")
```

```
------------------------------
Punto 4. Modelo logistico de pesca.
------------------------------
----------------------------------------
Población al final del Año 1: 416.00 peces
Población al final del Año 2: 433.18 peces
----------------------------------------
Punto de equilibrio (Estable)):   723.61 peces
Punto de equilibrio (Inestable): 276.39 peces
----------------------------------------
Interpretación: La población sobrevivirá y tenderá al equilibrio estable.
```